SOFTWARE TESTING

ASSIGNMENT



Name : Gauri Vinodrao Mahalle
Class : Msc 1st year (Sem –I)
Subject : Software Testing
Submitted to : Gawande Sir

1.What is URL manipulation attack? How do we safeguard our applications from these attacks? (4 MARKS)

Meaning of URL Manipulation Attack

A URL manipulation attack happens when an attacker changes some part of a web address to try to access information or functions that they are not permitted to use. A URL may contain a page name, a record number, a search term, or other parameters. If an application blindly trusts those values, an attacker may change them and receive another user's data or reach a restricted page.

For example, a website may show a customer's account at https://example.com/account?id=105. If the server checks only that the user is logged in, but does not check whether the account belongs to that user, the user might change the number to 106 and see another customer's account. This is a form of insecure direct object reference (IDOR), which can occur through URL manipulation.

Example of URL Manipulation



If the server trusts the changed ID without checking access, one user may see another user's information.

How can it happen?

1. The attacker observes a URL containing an identifier, such as a user ID, order number, or file name.
2. The attacker edits the parameter or path in the browser's address bar.
3. The server receives the modified request.
4. If the server does not verify the user's permissions, it may return protected information or perform an unauthorized action.

URL manipulation does not always mean a successful attack. A secure application validates every request and refuses access when the user is not authorized. The main weakness is trusting data supplied by the browser without server-side validation and access check.

Common forms of URL manipulation

1. **Changing an ID:** The user changes a customer, student, order, or invoice number in the URL to try to view someone else's record.
2. **Changing a role or privilege parameter:** A poorly designed application may use a URL value such as role=user. An attacker may try changing it to role=admin. Permissions must never depend only on a URL parameter.
3. **Changing a file path:** An attacker changes a file or folder path to attempt to access a file outside the permitted area. This may overlap with path traversal vulnerabilities.
4. **Modifying prices or quantities:** If an application relies on a price sent by the browser, a person may alter it. The server should calculate prices using trusted data.
5. **Bypassing page restrictions:** A person may type a restricted page address directly rather than using the website's menu. Hiding a link is not security; the server must enforce authorization.

Possible effects

- Exposure of private information such as names, addresses, marks, bills, or account details.
- Unauthorized changes to profiles, orders, or records.
- Financial loss when payment or price information is not verified correctly.
- Violation of privacy, loss of user trust, and damage to the organization's reputation.
- Legal or compliance problems if personal or confidential data is exposed.

Illustrative example: college portal

Suppose a student can open /result?studentId=2201. The student changes the ID to 2202. If the portal displays the second student's marks, the application has an authorization weakness. The correct behaviour is to verify on the server that the logged-in student is allowed to view the requested result. A teacher may have broader access, but that access must be checked using the teacher's authenticated identity and assigned permissions.

Important point

The issue is not simply that the URL can be edited—users can always edit a URL. The real problem is when the server accepts an unauthorized request. Security must be enforced on the server for every request and every protected object.

Ways to safeguard an application

Server-side authorization: For every request, check who the user is and whether that user is allowed to access the specific record or perform the action. Never rely only on a hidden button, menu, or client-side check.

Validate input: Check parameters against expected formats, types, lengths, and allowed values. For example, an ID should be a valid integer in the expected range. Validation alone is not a replacement for authorization.

Use secure session management: Authenticate users properly, use secure cookies, expire inactive sessions, and avoid exposing session tokens in URLs.

Avoid trusting client-supplied prices or roles: Calculate important values on the server and read permissions from trusted server-side records.

Use indirect references where appropriate: Random or opaque identifiers can make guessing harder, but they do not replace access checks.

Apply least privilege: Each user should receive only the permissions needed for their work.

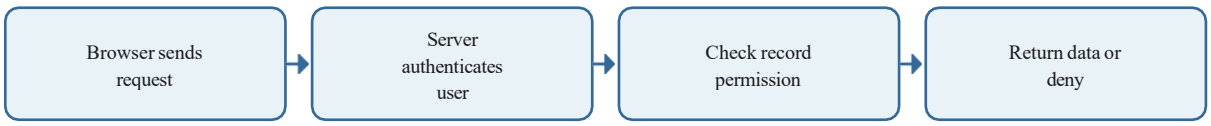
Log and monitor suspicious requests: Repeated attempts to access many IDs or restricted paths should be recorded and investigated, while avoiding unnecessary sensitive data in logs.

Test security regularly: Include negative tests, authorization tests, code reviews, and security testing before release and after important changes.

Example of a safe design

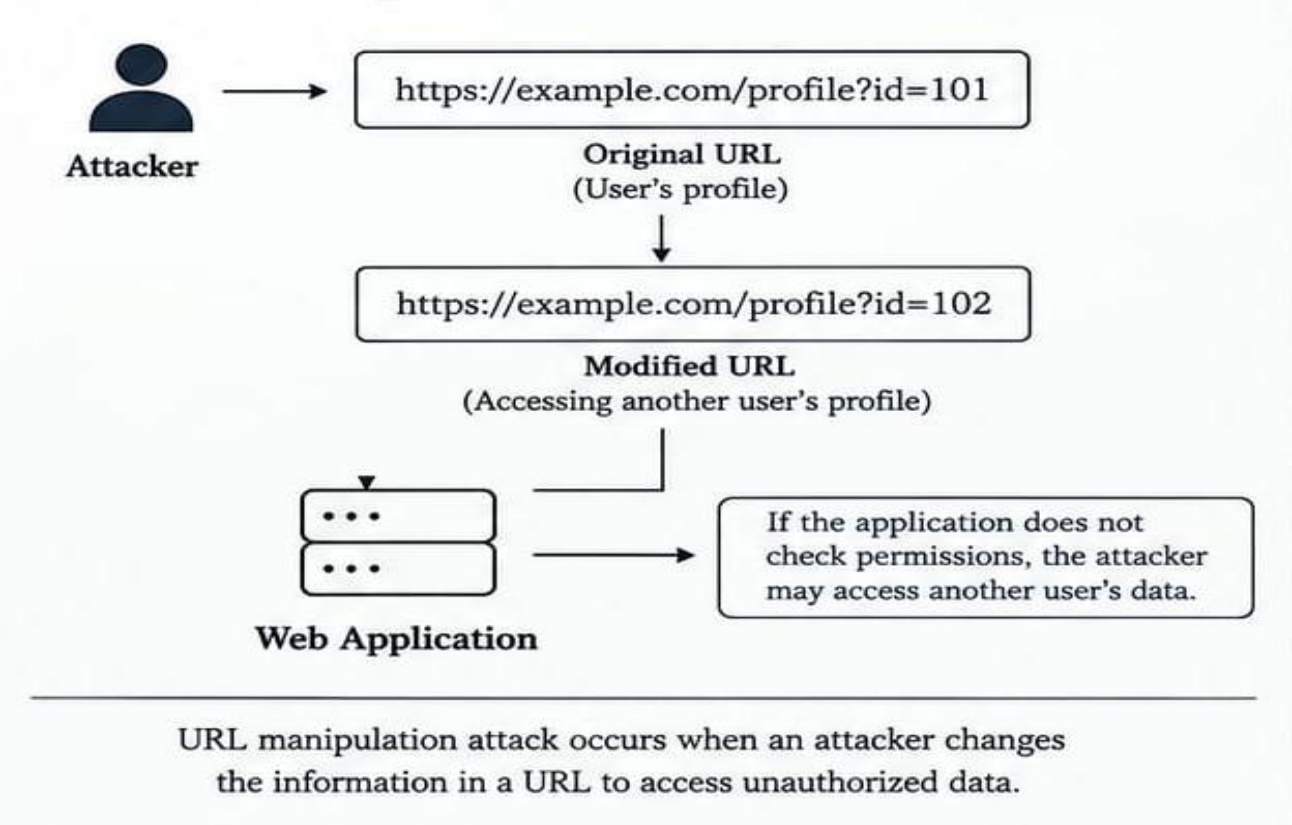
When a student requests a result, the server reads the authenticated student's identity from the session, loads the requested result, and checks whether that student owns the record. If the record belongs to someone else, the server returns an access-denied response, such as HTTP 403, or a carefully designed not-found response. The server should not return the record simply because its ID appears in the URL.

Safe Request Handling



URL manipulation is a security risk caused by changing URL paths or parameters to attempt unauthorized access. The most effective defence is server-side authorization on every request, supported by validation, secure sessions, least privilege, monitoring, and regular security testing. A secure application assumes that any value coming from the browser can be changed.

Diagram of URL Manipulation Attack



Dia.,URL Manipulation Attack

2. Describe testing a sample web application with an example. (6 MARKS)

Meaning and purpose

Testing a web application means checking its pages, features, inputs, links, calculations, access

permissions, speed, and behaviour on different devices and browsers. The purpose is to find defects before users depend on the application. A tester compares the actual result with the expected result and records any difference as a defect.

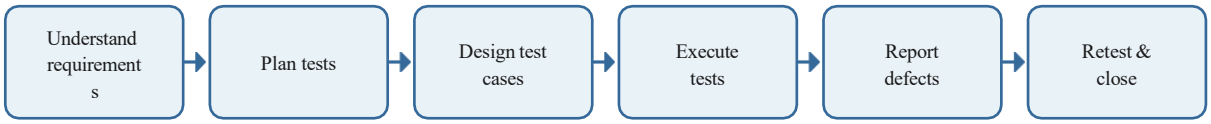
Sample application: Online Shopping Website

Consider a small online shopping website. A customer can register, log in, search for a product, add products to a cart, enter a delivery address, and place an order. An administrator can add products and update stock. This example is useful because it contains forms, calculations, user accounts, and protected actions.

Main requirements to test

- A new user can register with valid information.
- An existing user can log in with the correct password and cannot log in with an incorrect password.
- Product search returns relevant results and handles a search with no matches.
- The cart shows the correct item, quantity, and total price.
- The system rejects invalid quantities and unavailable stock.
- Checkout validates required fields and confirms an order only when the operation succeeds.
- A user can view only their own orders; an ordinary customer cannot access administrator functions.

Web Application Testing Cycle



Testing should be planned before execution. The tester first understands requirements, identifies risks, prepares test cases and test data, executes the checks, records results, reports defects, and retests after fixes.

Step-by-step testing procedure

Step 1: Study requirements

Read the requirement document or discuss the expected behaviour with the developer or product owner. Clarify what happens when the cart is empty, stock is unavailable, payment fails, or the user enters incorrect information. Clear requirements make it possible to decide whether a result is correct.

Step 2: Prepare the test environment

Use a test version of the website, a test database, and safe test accounts. Prepare sample products with different prices and stock quantities. Do not use real customer data or make real payments during ordinary testing. Record the browser, device, build version, and test environment so that a defect can be reproduced.

Step 3: Design test cases

Write clear steps and expected results. A test case should identify what is being checked, the starting conditions, the input data, the steps, and the expected outcome. Include valid inputs, invalid inputs, boundary values, and permission checks.

Step 4: Execute test cases

Follow the steps exactly and compare the actual output with the expected output. Mark each case as Pass, Fail, Blocked, or Not Run. If a case fails, capture useful evidence such as the error message, time, test data, and a screenshot where appropriate. Avoid placing passwords or private data in screenshots or logs.

Step 5: Report defects

A defect report should include a short title, environment, preconditions, steps to reproduce, expected result, actual result, severity, priority if used by the team, and evidence. For example: “Cart total does not update after quantity changes.” A good report is factual and easy for another person to repeat.

Step 6: Retest and regression test

After a developer fixes the defect, repeat the failed test to confirm the correction. Then run related tests to make sure the change has not broken another feature. This second activity is called regression testing.

Sample test cases and example results

Test Case	Test scenario	Test data / action	Expected result
TC-01	Valid login	Correct email and password	User reaches account page
TC-02	Invalid login	Correct email, wrong password	Login is rejected safely
TC-03	Search product	Search for “bag”	Relevant products are displayed
TC-04	Add to cart	Add one available item	Item and correct price appear
TC-05	Quantity boundary	Enter 0 or a negative quantity	Invalid quantity is rejected
TC-06	Checkout validation	Leave delivery address blank	Clear validation message appears
TC-07	Order privacy	Try opening another user's order	Access is denied
TC-08	Stock validation	Attempt to buy more than available	prevented or quantity limited

Detailed example: checking the cart total

Precondition: The user is logged in and a product costing 500 is available.

Steps: Open the product page, add one unit to the cart, open the cart, and inspect the subtotal. Increase the quantity to two and inspect the subtotal again.

Expected result: One unit gives a subtotal of 500; two units give a subtotal of 1,000, before any delivery fee, tax, or discount that the requirements specify.

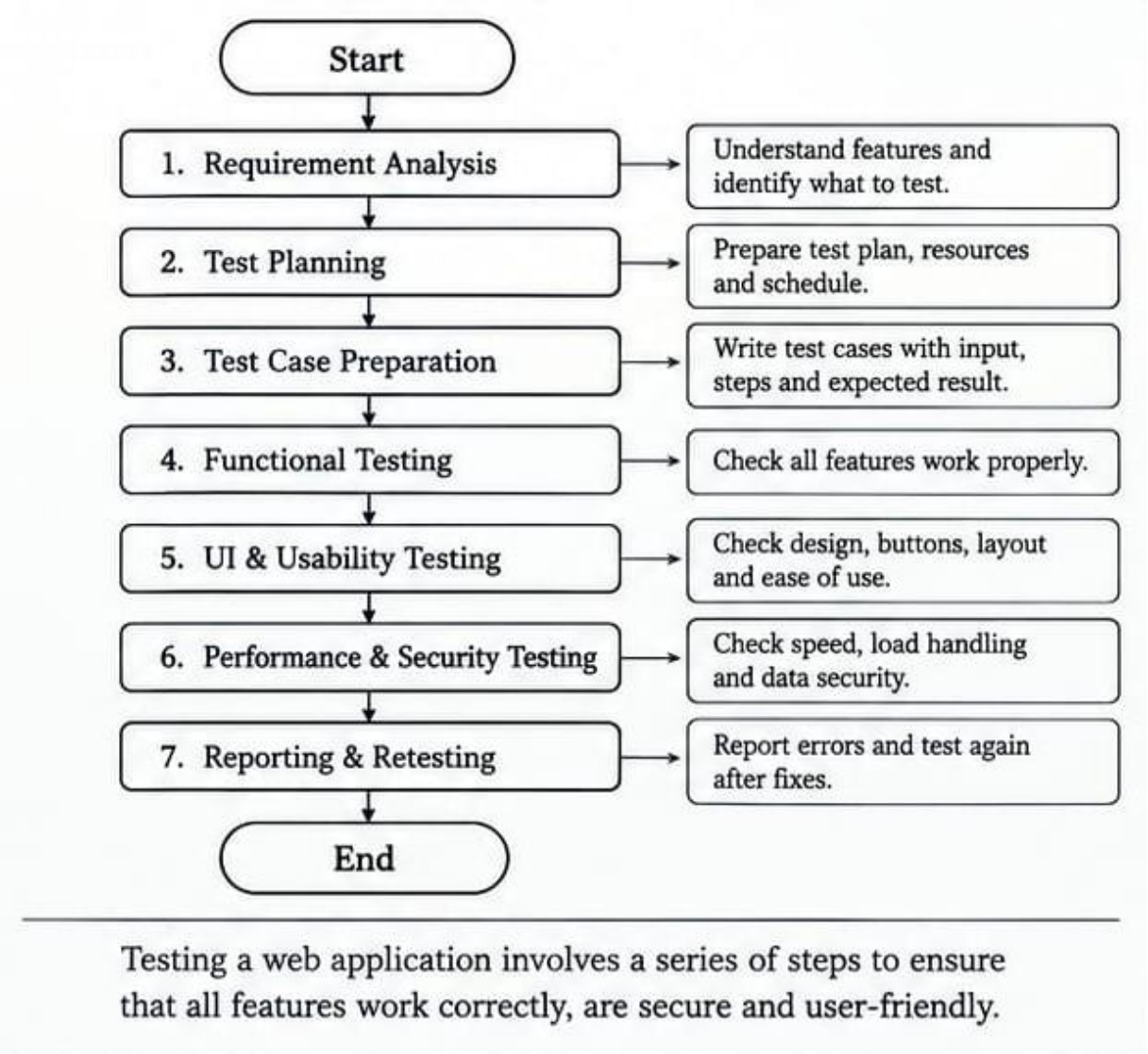
Possible defect: If the displayed subtotal remains 500 after increasing the quantity to two, the calculation or page update may be incorrect. The tester records the actual value and reports the issue.

Types of testing used in this example

Functional testing checks whether login, search, cart, and checkout work. Usability testing checks whether the instructions and error messages are understandable. Compatibility testing checks the website in different supported browsers and screen sizes. Performance testing checks response time under expected load. Security testing checks authentication, authorization, input handling,

and protection of sensitive data.

Diagram of sample web application



Dia.,sample web application

3. What is Test case tracking? Explain why it is important in software testing.

(4 MARKS)

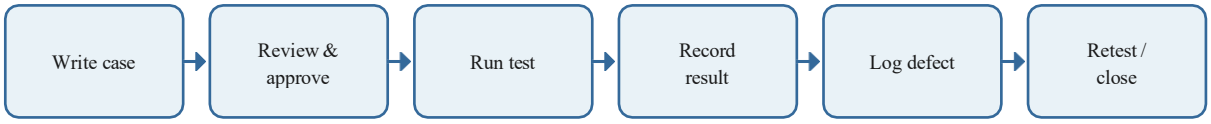
Meaning of test case tracking

Test case tracking is the process of monitoring the progress and status of test cases throughout a testing

project. It tells the team which test cases have been written, reviewed, executed, passed, failed, blocked, or not yet run. It may be managed in a spreadsheet, a test management tool, or a defect tracking system.

A test case is not complete simply because it has been written. The team also needs to know whether it has been executed, what result was observed, whether a defect was raised, and whether the test passed after the defect was fixed. Tracking keeps this information organized and visible.

Test Case Tracking Workflow



Typical statuses

Not Run: The test has not yet been executed.

In Progress: The tester is currently performing it.

Pass: Actual behaviour matches the expected result.

Fail: Actual behaviour does not match the expected result.

Blocked: Testing cannot continue because of an issue such as an unavailable service or missing test data.

Retest: The test is repeated after a defect is fixed.

Closed: The case or related issue has reached the team's agreed final status.

Simple example

Suppose a team has 100 test cases for a college admission portal. Tracking may show 60 passed, 10 failed, 5 blocked, and 25 not run. This gives the team a clear picture of progress. The failed cases can be linked to defect reports, while blocked cases can be assigned to the person who can remove the blocker.

Importance and benefits of tracking

Why test case tracking is important

1. **Measures progress:** The team can see how much testing has been completed and how much work remains.
2. **Improves accountability:** It is clear who is responsible for executing or reviewing a test case.
3. **Helps find defects early:** Failed tests are visible and can be investigated before release.
4. **Supports release decisions:** Managers can review pass rates, critical failures, unresolved defects, and untested areas before deciding whether the application is ready.
5. **Prevents missed testing:** Tracking helps ensure that important requirements and high-risk features are not forgotten.
6. **Supports retesting and regression testing:** The team can identify which tests must be repeated after a code change.
7. **Improves communication:** Testers, developers, project managers, and business stakeholders can work from the same status information.
8. **Provides evidence and traceability:** Results can be connected to requirements, defects, builds, and release records.

Useful information to track

A tracking record may contain the test case ID, title, linked requirement, priority, test environment, tester, execution date, result, defect ID, retest status, and comments. Teams should use consistent status definitions so that reports mean the same thing to everyone.

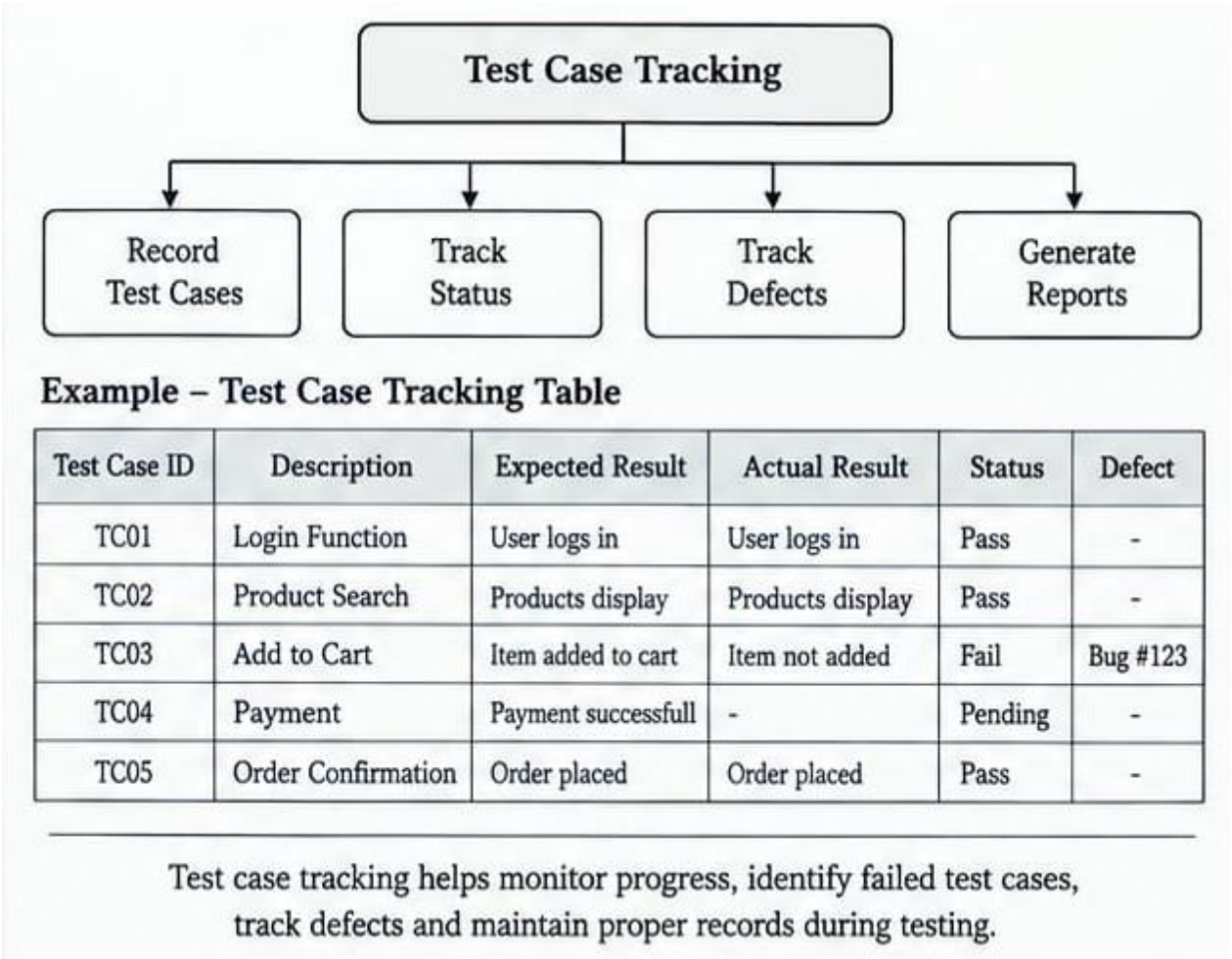
Example progress calculation

If 80 out of 100 test cases have been executed, execution progress is 80%. If 68 of those 80 executed cases passed, the pass rate among executed cases is 85%. These figures answer different questions: execution progress tells us how much work has been attempted, while pass rate tells us how many executed tests succeeded. Neither figure alone proves that the application is ready; the importance and severity of failures also matter.

Good practices

Update results soon after execution, link failed cases to defects, keep test data and evidence safe, review blocked tests, and avoid marking a test as passed without checking its expected result. Reports should be accurate rather than made to look positive.

Diagram of Test case tracking



Example – Test Case Tracking Table

Test Case ID	Description	Expected Result	Actual Result	Status	Defect
TC01	Login Function	User logs in	User logs in	Pass	-
TC02	Product Search	Products display	Products display	Pass	-
TC03	Add to Cart	Item added to cart	Item not added	Fail	Bug #123
TC04	Payment	Payment successfull	-	Pending	-
TC05	Order Confirmation	Order placed	Order placed	Pass	-

Test case tracking helps monitor progress, identify failed test cases, track defects and maintain proper records during testing.

Dia.,of Test case tracking

4.What is a test case? What are various types of test cases? (6MARKS)

Definition of a test case

A test case is a written set of conditions, inputs, steps, and expected results used to check one particular behaviour of software. It explains what the tester should do and what should happen if the application works correctly. Test cases make testing repeatable: different testers can follow the same instructions and compare their results.

For example, a login test case may specify that a registered user enters a valid email address and password, clicks Login, and should then reach the account page. Another test case checks that an incorrect password is rejected and that the application does not reveal sensitive account information.

Main parts of a test case

Test case ID: A unique label, such as TC-LOGIN-01.

Title or objective: What the test checks.

Preconditions: What must be true before the test begins, such as having a registered account.

Test data: Inputs needed for the test.

Steps: The actions the tester performs, in order.

Expected result : The correct outcome according to requirements.

Actual result: What really happened during execution.

Status: Pass, Fail, Blocked, or Not Run.

Priority / notes: How important the test is and any useful observations.Sample test case format

Test Case ID	**TC-LOGIN-01**
Objective	Verify login with valid credentials
Precondition	User account already exists
Test data	Valid registered email and password
Steps	1. Open login page 2. Enter credentials 3. Click Login
Expected result	User reaches the account dashboard
Actual result	Fill in during execution
Status	Not Run until tested

Types of test cases – functional and input-based

There are many ways to classify test cases. The exact names may differ between organizations, but the following types are widely used.

1. Functional test cases

These check whether a feature performs the function described in the requirements. Examples include checking login, registration, search, add-to-cart, payment confirmation, password reset, and logout. A functional test compares the actual behaviour with the required behaviour.

2. Positive test cases

These use valid input and the expected sequence of actions. Example: enter a registered email address and correct password, then verify that login succeeds. Positive tests confirm that normal users can complete their tasks.

3. Negative test cases

These use invalid, missing, or unexpected input to check whether the application handles errors safely. Example: submit a login form with a wrong password or leave a required field empty. The application should give a clear message and should not crash or expose private information.

4. Boundary value test cases

These check values at the edges of an allowed range, because defects often occur at boundaries. If an age field accepts values from 18 to 60, test 17, 18, 19, 59, 60, and 61. This checks values just below, at, and just above the permitted limits.

5. Equivalence partition test cases

Inputs are divided into groups expected to behave similarly. For a field accepting numbers from 1 to 10, one valid group is 1–10 and invalid groups include values below 1 and above 10. A representative value from each group can reduce the number of tests while maintaining useful coverage.

6. Validation test cases

These check field rules, formats, required values, and length limits. Examples include an invalid email format, a password shorter than the minimum length, letters in a numeric-only field, or a file type that is not permitted. Validation should be applied on the server as well as in the browser where appropriate.

Types of test cases – quality, security and change-related

7. Usability test cases

These check whether the application is understandable and convenient to use. They may examine navigation, labels, error messages, readability, keyboard use, and whether users can complete a task without confusion. For example, a checkout page should clearly identify which required field has been missed.

8. Compatibility test cases

These verify that the application works in the browsers, operating systems, devices, and screen sizes that the project supports. A page may look correct on a laptop but have overlapping buttons on a mobile screen, so responsive layouts should be checked.

9. Performance test cases

These evaluate response time, stability, and behaviour under expected or heavy workloads. For example, a team may check whether a product search remains responsive when many users search at the same time. Load testing, stress testing, and endurance testing are related performance activities.

10. Security test cases

These check whether the application protects accounts, data, and restricted functions. Examples include testing access control, session expiry, password reset, input handling, and whether one user can access another user's order by changing an identifier in a URL. Security tests should be performed only on systems where the tester has permission.

11. Regression test cases

These are repeated after a change to check that existing features still work. For example, after modifying the shopping-cart calculation, rerun tests for quantity changes, discounts, taxes, checkout, and order totals.

12. Integration test cases

These check whether connected components work together. For example, after an order is placed, the order service, inventory service, and confirmation email service should exchange the correct information.

13. End-to-end test cases

These verify a complete user journey from beginning to end. In an online shop, an end-to-end test may cover login, search, add to cart, checkout, viewing the order in the account page.

More test case categories and selection

14. Smoke test cases

Smoke tests are a small set of important checks performed on a new build to decide whether it is stable enough for deeper testing. Examples include opening the site, logging in, loading the main page, and checking that a key workflow starts successfully.

15. Sanity test cases

Sanity testing is a focused check of a particular change or area after a small update. For example, if a search filter has been corrected, the tester checks the filter and closely related behaviour before running a wider regression suite.

16. Database test cases

These verify that information is stored, updated, retrieved, and deleted correctly, according to requirements. Examples include checking that an order total in the database matches the confirmed order and that deleting a test record does not damage unrelated records.

17. Accessibility test cases

These check whether people with different abilities can use the application. Examples include keyboard-only navigation, meaningful labels for form controls, visible focus, readable contrast, and appropriate alternative text for informative images.

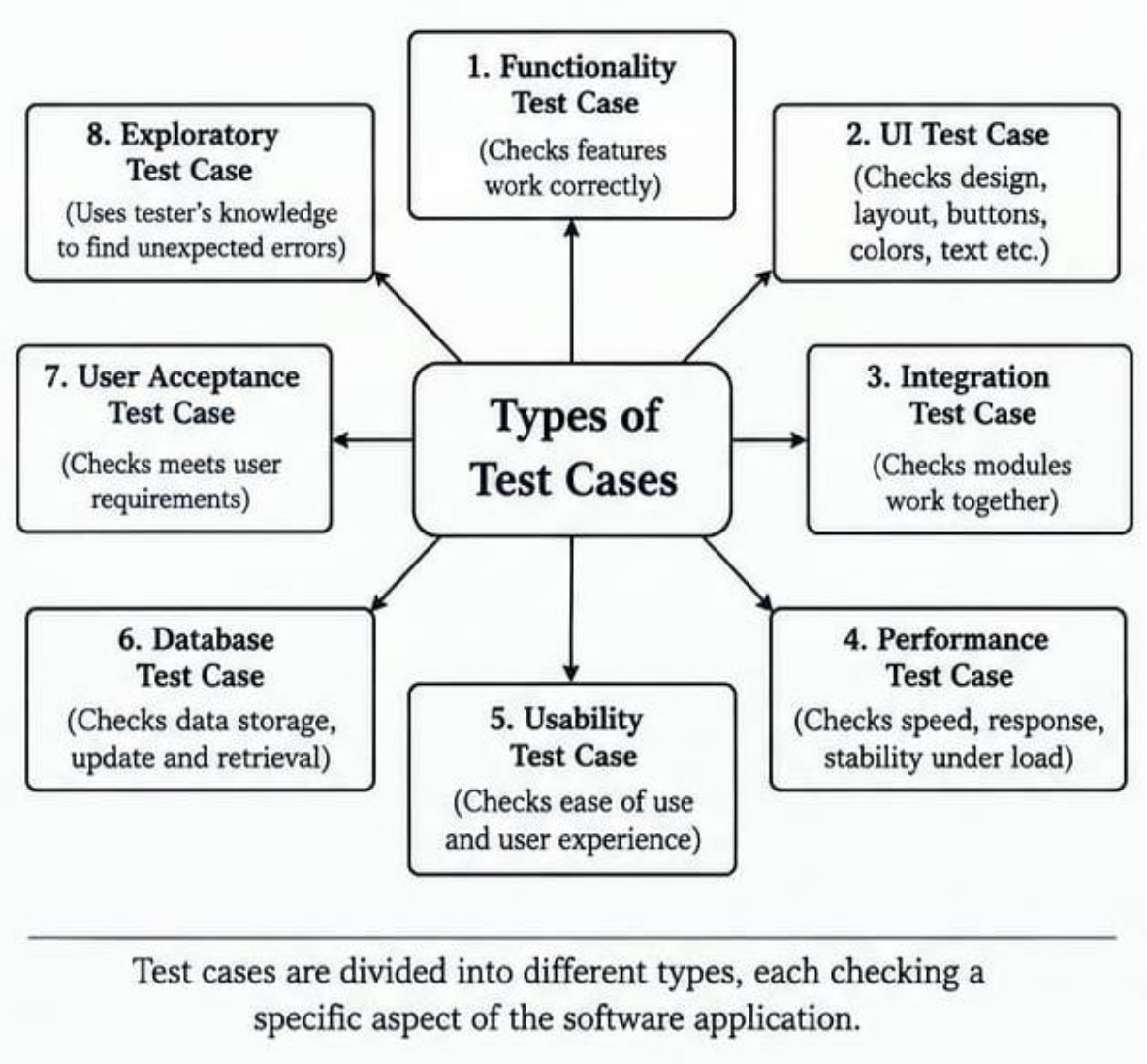
18. Recovery and reliability test cases

These check how the application behaves after interruptions or failures. For example, if a network connection drops during checkout, the user should receive a clear status and should not be charged twice because of a repeated request.

How to choose test cases

A tester should begin with important business requirements and high-risk features. Test cases should cover ordinary user journeys, invalid inputs, boundaries, security permissions, different supported devices, and likely failure conditions. When time is limited, prioritize cases based on impact, likelihood of failure, and the number of users affected. Keep test cases updated when requirements change.

Diagram of Types of Test Cases



Dia.,of Types of Test Cases